

Assignment: Stored Procedures & UDFs — Hands-on & Scenario-Based Practice

Note: Work inside `college_db.academics`. Write and test every function/procedure in pgAdmin's Query Tool. Use `SELECT` to test functions and `CALL` to test procedures. Observe real error messages.

Setup Reference

Use the existing college schema tables (students, departments, courses, enrollments). Ensure you have varied CGPA values and some graded enrollments.

Q1. Scalar Function — Grade to Points

Create a function `academics.grade_to_points(p_grade CHAR(2))` that returns NUMERIC points: A+=10, A=9, B+=8, B=7, C=6, F=0, anything else NULL. Test it with `SELECT academics.grade_to_points('A+');`

-- Write CREATE FUNCTION and test SELECT here:

```
CREATE OR REPLACE FUNCTION academics.grade_to_points(
    p_grade CHAR(2)
)
RETURNS NUMERIC
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN CASE p_grade
        WHEN 'A+' THEN 10
        WHEN 'A' THEN 9
        WHEN 'B+' THEN 8
        WHEN 'B' THEN 7
        WHEN 'C' THEN 6
        WHEN 'F' THEN 0
        ELSE NULL
    END;
END;
$$;
```

Q2. Function with Default Parameter

Create `academics.cgpa_category(p_cgpa NUMERIC, p_pass_mark NUMERIC DEFAULT 6.0)` that returns TEXT: 'Excellent' (>=9), 'Good' (>=7), 'Average' (>= pass_mark), 'Fail' (below pass_mark), 'Unknown' (NULL cgpa). Test with and without the second argument.

-- Write CREATE FUNCTION and test SELECTs here:

```
CREATE OR REPLACE FUNCTION academics.cgpa_category(
    p_cgpa NUMERIC,
    p_pass_mark NUMERIC DEFAULT 6.0
)
RETURNS TEXT
```

```
LANGUAGE plpgsql
AS $$
BEGIN

    IF p_cgpa IS NULL THEN
        RETURN 'Unknown';

    ELSIF p_cgpa >= 9 THEN
        RETURN 'Excellent';

    ELSIF p_cgpa >= 7 THEN
        RETURN 'Good';

    ELSIF p_cgpa >= p_pass_mark THEN
        RETURN 'Average';

    ELSE
        RETURN 'Fail';
    END IF;

END;
$$;
```

Q3. Set-Returning Function

Create `academics.get_dept_students(p_dept_id INTEGER)` that RETURNS TABLE(`student_id` INTEGER, `full_name` VARCHAR, `cgpa` NUMERIC). It should return all students of that department ordered by `cgpa` DESC. Test with `SELECT * FROM academics.get_dept_students(1);`

-- Write CREATE FUNCTION and test SELECT here:

```
CREATE OR REPLACE FUNCTION academics.get_dept_students(
    p_dept_id INTEGER
)
RETURNS TABLE (
    student_id INTEGER,
    full_name VARCHAR,
    cgpa NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN

    RETURN QUERY
    SELECT
        s.student_id,
        s.full_name,
```

```

        s.cgpa
    FROM academics.students s
    WHERE s.dept_id = p_dept_id
    ORDER BY s.cgpa DESC;

END;
$$;

```

```

—Test
SELECT *
FROM academics.get_dept_students(1);

```

Q4. Function — Calculate Student GPA

Create academics.calc_gpa(p_student_id INTEGER) that computes the average grade points of all graded enrollments for that student (use your grade_to_points function). Return NULL if the student has no graded enrollments. Test for at least two students.

-- Write CREATE FUNCTION and test SELECTs here:

```

CREATE OR REPLACE FUNCTION academics.calc_gpa(
    p_student_id INTEGER
)
RETURNS NUMERIC
LANGUAGE plpgsql
AS $$
DECLARE
    v_gpa NUMERIC;
BEGIN

    SELECT AVG(
        academics.grade_to_points(e.grade)
    )
    INTO v_gpa
    FROM academics.enrollments e
    WHERE e.student_id = p_student_id
    AND e.grade IS NOT NULL;

    RETURN v_gpa;
END;
$$;

```

Q5. Basic Procedure — Enroll Student

Create a procedure academics.enroll_student(p_student_id INTEGER, p_course_id INTEGER, p_grade CHAR(2) DEFAULT NULL) that inserts a row into enrollments. Test it with CALL.

-- Write CREATE PROCEDURE and CALL here:

```

CREATE OR REPLACE PROCEDURE academics.enroll_student(
    p_student_id INTEGER,

```

```

    p_course_id INTEGER,
    p_grade CHAR(2) DEFAULT NULL
)
LANGUAGE plpgsql
AS $$
BEGIN

    INSERT INTO academics.enrollments (
        student_id,
        course_id,
        grade
    )
    VALUES (
        p_student_id,
        p_course_id,
        p_grade
    );

END;
$$;

```

—Test

```
CALL academics.enroll_student(1, 2);
```

Q6. Procedure with Validation & EXCEPTION

Enhance (or recreate) the enroll procedure so that:

- It raises an EXCEPTION if the student_id does not exist
- It raises an EXCEPTION if the course_id does not exist
- It catches unique_violation and raises a NOTICE that the student is already enrolled

Test both success and failure paths.

-- Write CREATE PROCEDURE and test CALLs here:

```

CREATE OR REPLACE PROCEDURE academics.enroll_student(
    p_student_id INTEGER,
    p_course_id INTEGER,
    p_grade CHAR(2) DEFAULT NULL
)
LANGUAGE plpgsql
AS $$
BEGIN

```

```

    IF NOT EXISTS (
        SELECT 1

```

```
FROM academics.students
WHERE student_id = p_student_id
) THEN

RAISE EXCEPTION
    'Student with ID % does not exist.',
    p_student_id;

END IF;

IF NOT EXISTS (
    SELECT 1
    FROM academics.courses
    WHERE course_id = p_course_id
) THEN

RAISE EXCEPTION
    'Course with ID % does not exist.',
    p_course_id;

END IF;

BEGIN

INSERT INTO academics.enrollments (
    student_id,
    course_id,
    grade
)
VALUES (
    p_student_id,
    p_course_id,
    p_grade
);

EXCEPTION
    WHEN unique_violation THEN
RAISE NOTICE
    'Student % is already enrolled in course %.',
```

```

    p_student_id,
    p_course_id;

```

```

END;
END;
$$;

```

Q7. Procedure with OUT Parameters

Create academics.dept_summary(p_dept_id INTEGER, OUT student_count INTEGER, OUT avg_cgpa NUMERIC) that fills the OUT parameters with the count of students and average CGPA for that department. Test with CALL.

-- Write CREATE PROCEDURE and CALL here:

Q8. Procedure — Transfer Student

Create academics.transfer_student(p_student_id INTEGER, p_new_dept_id INTEGER) that:

- Validates both student and new department exist
- Updates students.dept_id
- Raises NOTICE on success
- Raises EXCEPTION on any validation failure

Test a successful transfer and a failure case.

-- Write CREATE PROCEDURE and test CALLs here:

Q9. Function Using Control Structures

Create academics.letter_from_cgpa(p_cgpa NUMERIC) that returns a letter grade using IF/ELSIF:

>= 9 → 'A+', >= 8 → 'A', >= 7 → 'B+', >= 6 → 'B', >= 5 → 'C', else 'F'. Handle NULL.

-- Write CREATE FUNCTION and test SELECTs here:

Q10. Set-Returning Function with Join Logic

Create academics.student_course_report(p_student_id INTEGER) that RETURNS TABLE(course_name VARCHAR, grade CHAR(2), points NUMERIC). It should return all courses the student is enrolled in along with grade and grade points (reuse grade_to_points). Test it.

-- Write CREATE FUNCTION and test SELECT here:

Q11. Procedure — Bulk Grade Update

Create a procedure academics.bulk_update_grade(p_course_id INTEGER, p_old_grade CHAR(2), p_new_grade CHAR(2)) that updates all enrollments for that course from old_grade to new_grade. Raise NOTICE with the number of rows updated. Test it.

-- Write CREATE PROCEDURE and CALL here:

Q12. Drop & Document

- List the functions and procedures you created (query pg_proc or just note the names).
- DROP any experimental ones you no longer need. Keep the useful ones (grade_to_points, calc_gpa, enroll_student, etc.) for upcoming assignment.
- In 2-3 sentences explain when you would choose a Function over a Procedure (and vice versa).

-- (a) list, (b) DROP statements, (c) explanation:

Note: *Keep the useful functions/procedures. Next we (Cursors & Triggers) will call them and will also introduce automatic logic that fires on data changes.*